

TP01 - Agentes de Busca em Ambientes de Labirinto

Disciplina: CSI457 - Agentes de Inteligencia Artificial

Instituicao: Universidade Federal de Ouro Preto (UFOP)

Periodo: 2026/1

Entrega: Semanas 1, 2 e 3

Sumario

1. [Introducao](#1-introducao)
 2. [Modelagem PEAS](#2-modelagem-peas)
 3. [Formulacao Formal do Problema](#3-formulacao-formal-do-problema)
 4. [Algoritmos Implementados](#4-algoritmos-implementados)
 5. [Arquitetura do Sistema](#5-arquitetura-do-sistema)
 6. [Resultados e Experimentos](#6-resultados-e-experimentos)
 7. [Visualizacao](#7-visualizacao)
 8. [Auditoria de Uso de IA](#8-auditoria-de-uso-de-ia)
 9. [Conclusao](#9-conclusao)
-

1. Introducao

Este trabalho pratico implementa um sistema de agentes inteligentes capazes de resolver problemas de navegacao em labirintos usando tres abordagens de busca:

- * Semana 1 - Busca Classica: o mapa e completamente conhecido antes da execucao. O agente planeja o caminho inteiro antes de se mover.
- * Semana 2 - Busca Local: o agente otimiza a ordem de visitacao de pontos de coleta sem manter uma fronteira global, usando Hill-Climbing, Simulated Annealing e Algoritmo Genetico.
- * Semana 3 - Busca Online: o mapa e desconhecido. O agente percebe apenas as celulas adjacentes a cada passo e constroi seu modelo interno do ambiente durante a execucao.

O projeto reutiliza e estende componentes da ATV02, reorganizando os algoritmos em modulos independentes e adicionando suporte a estados estendidos (coletas), busca local e busca online com percepcao parcial.

2. Modelagem PEAS

A modelagem PEAS descreve o agente em suas tres configuracoes ao longo das semanas.

2.1 Semana 1 - Busca Classica

PEAS	Descricao
Performance	Encontrou solucao? · Custo do caminho (m)
Environment	Grade discreta com paredes, espacos livr
Actuators	Mover cima (i-1,j) · Mover baixo (i+1,j)
Sensors	Mapa completo (matriz de paredes inteira)

Tipo de agente: baseado em objetivos com modelo interno completo do ambiente.

Estrategia: planeja o caminho completo antes de executar qualquer acao.

2.2 Semana 2 - Busca Local

PEAS	Descricao
Performance	Qualidade da solucao $f(s) = -h(s)$ (maxim
Environment	Mesma grade da Semana 1. Totalmente obse
Actuators	Mesmos 4 movimentos ortogonais. O agente
Sensors	Valores de f para cada vizinho imediato

Tipo de agente: baseado em utilidade - maximiza $f(s)$ sem objetivo global.

2.3 Semana 3 - Busca Online

PEAS	Descricao
Performance	Encontrou o objetivo B? · Custo total pe
Environment	Grade desconhecida a priori. Parcialment
Actuators	Mesmos 4 movimentos ortogonais · Backtra
Sensors	Percepcao local a cada passo: $\sigma(pos)$

Tipo de agente: baseado em objetivos com modelo interno parcial e incremental.

Estrategia: intercala percepcao -> atualizacao de M -> acao a cada passo.

2.4 Quadro Comparativo PEAS

PEAS	Busca Classica	Busca Local	Busca Online
Performance	custo + exp + tempo + memoria	$f(s)$ local (qualidade)	custo + backtr. + cobertura
Observabilidade	Total	Total	Parcial (raio=1)
Determinismo	Sim	Sim	Sim
Estaticidade	Sim	Sim	Sim
Actuators	4 movimentos	4 movimentos	4 mov. + volta
Sensors	Mapa completo (pre-busca)	Vizinhos + f (por passo)	Celulas adj. $r=1$ (por passo)
Tipo de agente	Baseado em objetivos	Baseado em utilidade	Obj. + modelo parcial

3. Formulacao Formal do Problema

3.1 Notacao Unificada

O problema e formulado como $P = \langle S, A, T, s_0, G, c \rangle$ onde:

- * S - espaco de estados
- * A - acoes disponiveis
- * T - funcao de transicao $T: S \times A \rightarrow S$
- * s_0 - estado inicial
- * G - condicao de objetivo $G \subseteq S$
- * c - custo de acao $c: A \rightarrow \mathbb{R}^+$

3.2 Semana 1A - Problema Simples (sem coletas)

```

S = { (i,j) | 0 <= i < H, 0 <= j < W, labirinto[i][j] != parede }

A = { cima, baixo, esquerda, direita }

T = deterministico e total:
    T((i,j), cima)      = (i-1, j)   se válido
    T((i,j), baixo)     = (i+1, j)   se válido
    T((i,j), esquerda) = (i, j-1)   se válido
    T((i,j), direita)  = (i, j+1)   se válido
    T((i,j), a)         = nulo       caso contrário

s0 = coordenadas do marcador 'A'
G = { coordenadas do marcador 'B' }
c = c(a) = 1 para toda ação a em A (custo uniforme)

```

Classificacao: Totalmente observavel · Deterministico · Estatico · Discreto · Agente unico

Complexidade (grade $m \times n$, fator de ramificacao $b \leq 4$):

- * Tempo: $O(b^d)$ onde d = profundidade da solucao
- * Espaco: $O(b^d)$ BFS | $O(db)$ DFS | $O(b^d)$ A

3.3 Semana 1B - Problema com Coletas

O agente deve visitar todos os pontos C antes de atingir B . A posicao sozinha nao e suficiente - e preciso registrar quais coletas ainda faltam. Solucao: estado estendido.

```

S' = { (pos, C_r) | pos em S, C_r ⊆ C }
    onde C = conjunto fixo de pontos de coleta do mapa
    |S'| = |S| × 2^|C|

s0' = ( posição_A, frozenset(C) )      <- nenhuma coleta visitada
G' = { ( posição_B, {} ) }             <- B atingido com todas as coletas

T' = T'( (pos, C_r), a ) = ( pos', C_r \ {pos'} )
    onde pos' = T(pos, a)

Heurística admissível para A*:
    h((pos, C_r)) = max{ manhattan(pos, d) | d em C_r ∪ {B} }
    Nunca superestima: agente precisa ao menos chegar ao destino mais distante.

```

3.4 Semana 2 - Busca Local

```

S = { (pos, C_r) } (mesmo espaço estendido de 1B)
N(s) = vizinhos imediatos de s
f(s) = -h(s)          <- função a MAXIMIZAR
      = -max{ manhattan(pos, d) | d em C_r U {B} }

Hill-Climbing:
  s <- s0
  enquanto f(melhor vizinho de s) > f(s):
    s <- argmax_{s' em N(s)} f(s')
  retorna s

Simulated Annealing:
  P(aceitar s') = 1          se Df > 0 (melhora)
                  e^(Df / T_k) se Df <= 0 (piora)
  onde Df = f(s') - f(s) e T_k -> 0 (resfriamento geométrico)

```

3.5 Semana 3 - Busca Online

```

Estado interno: s = (pos, M)
  pos -- posição atual (conhecida pelo agente)
  M   -- modelo interno do mapa construído até o momento
        M ? S, cresce monotonicamente a cada percepção

Percepção: sigma(pos) = { (pos', tipo) | pos' adjacente a pos }
              tipo em { livre, parede, início, objetivo, coleta }

Atualização: M <- M U sigma(pos) a cada novo passo

Replanning A*: a cada passo, executa A* no mapa interno M
  e replaneja quando uma parede previamente desconhecida é revelada.

Online DFS: mantém pilha de backtrack e lista de ações não tentadas
  por posição; explora sistematicamente sem memória global.

```

3.6 Quadro Comparativo

Propriedade	Busca Classica	Busca Local	Busca Online
Observab.	Total	Total	Parcial (raio 1)
Memoria	$O(b^d)$	$O(1)$	$O(\text{visitados})$
Completo	Sim (BFS/A*)	Nao (HC)	Sim (Replanning A*)
Otimo	Sim (UCS/A*)	Nao	Nao (1a travessia)
Estado	pos ou ext.	pos ou ext.	(pos, mapa interno)
Planejamento	Antes de agir	Antes de agir	Durante a acao

4. Algoritmos Implementados

4.1 Busca Classica (Semana 1)

BFS - Breadth-First Search

- * Fronteira como fila FIFO, exploracao por niveis
- * Completo: Sim · Otimo: Sim (custos uniformes)
- * Complexidade: Tempo $O(b^d)$ · Espaco $O(b^d)$

DFS - Depth-First Search

- * Fronteira como pilha LIFO, exploracao em profundidade
- * Completo: Nao (pode ciclar) · Otimo: Nao
- * Complexidade: Tempo $O(b^m)$ · Espaco $O(b \cdot m)$

UCS - Uniform Cost Search

- * Expande no com menor custo acumulado $g(n)$, min-heap
- * Completo: Sim · Otimo: Sim
- * Complexidade: $O(b^{(1+C^*/e)})$

Busca Gulosa

- * Expande por menor $h(n)$ = distancia Manhattan ao objetivo
- * Completo: Nao · Otimo: Nao
- * Rapida mas nao garante solucao otima

A* (A-Star)

- * Expande por menor $f(n) = g(n) + h(n)$
- * Completo: Sim · Otimo: Sim (h admissivel)
- * Heuristica Manhattan e admissivel: nunca superestima

4.2 Busca Local (Semana 2)**Hill-Climbing (Steepest Ascent)**

- * Representacao: permutacao de indices das coletas (ordem de visitacao)
- * Vizinhanca: troca de dois elementos (swap) - gera $k \cdot (k-1)/2$ vizinhos
- * Reinicializacao aleatoria para estimar metricas estatisticas
- * Limitacao: fica preso em maximos locais e platos

Simulated Annealing

- * Aceita solucoes piores com probabilidade $e^{-(\Delta f/T_k)}$, decrescente
- * Esquema de resfriamento geometrico: $T_{k+1} = \alpha \cdot T_k$
- * Parametros: temperatura inicial, taxa de resfriamento, iteracoes por temperatura

Algoritmo Genetico

- * Populacao de permutacoes das coletas
- * Operadores: selecao por torneio, cruzamento de ordem (OX), mutacao por swap
- * Evolui por geracoes; mantem o melhor individuo (elitismo)

4.3 Busca Online (Semana 3)

Replanning A*

- * Executa A* no mapa interno (tratando desconhecido como livre)
- * Ao revelar uma parede, descarta o plano e replaneja a partir da posicao atual
- * Razao online/offline = custo percorrido / custo otimo offline (A*)

Online DFS

- * Mantem lista de acoes nao tentadas por posicao e pilha de backtrack
- * Explora sistematicamente sem fronteira global
- * Garante completude em ambientes finitos e conectados

5. Arquitetura do Sistema

5.1 Estrutura de Diretorios

```
tp01/
+-- src/
|   +-- main.py           # Menu interativo (3 modos)
|   +-- labirinto.py      # LabirintoBusca, LabirintoComColetas,
|   |                     # LabirintoOnline, No, ResultadoBusca
|   +-- exibir.py         # Renderização no terminal
|   +-- gerar_html.py     # Animacao web (HTML/CSS/JS)
|   +-- formulacao_formal.py # Documentacao da formulacao
|   +-- agente.py         # Modelagem PEAS + classe AgenteLabirinto
|   +-- experimentos.py   # Suite de testes com CSV
|   +-- buscas/
|   |   +-- classicas/    # bfs, dfs, ucs, gulosa, a_estrela
|   |   +-- local/        # hill_climbing, simulated_annealing,
|   |   |                 # genetic_algorithm, distancias, resultado_local
|   |   +-- online/       # replanning_a_estrela, online_dfs
|   +-- mapas/
|   |   +-- lab1.txt      (simples, sem coletas)
|   |   +-- lab2.txt      (com 2 pontos de coleta)
|   |   +-- lab3.txt      (serpentino)
|   |   +-- lab4.txt      (complexo)
|   |   +-- lab5.txt      (grande)
|   |   +-- lab6.txt      (busca local)
|   |   +-- lab7.txt      (6 coletas ? mapa oculto do professor)
+-- RELATORIO_FINAL.md
+-- uso_ia.md
+-- peas.md
```

5.2 Interface Protocol

Para polimorfismo sem heranca nominal, usamos Protocol:

```
@runtime_checkable
class Labirinto(Protocol):
    inicio: Any
    objetivo: Any
    def vizinhos(self, estado) -> List[Tuple]: ...
```

```
def h(self, estado) -> float: ...
def reconstruir(self, no: No) -> Tuple[List, List]: ...
```

Tanto LabirintoBusca quanto LabirintoComColetas satisfazem o protocolo, permitindo que todos os algoritmos classicos funcionem com ambos sem modificacao.

5.3 LabirintoOnline

Simula percepcao parcial: o mapa real e oculto; o agente constroi um mapa interno (None = desconhecido, False = livre, True = parede) via chamadas a perceber(pos) a cada passo.

```
class LabirintoOnline:
    def perceber(self, pos) -> (novos_livres, novas_paredes)
    def mover(self, pos, acao) -> nova_pos ou None
    def vizinhos_livres_internos(self, pos) -> ações válidas no mapa interno
    def h_interna(self, pos) -> Manhattan ao objetivo
```

6. Resultados e Experimentos

6.1 Busca Classica - Lab1 (9x9, sem coletas)

Algoritmo	Sucesso	Custo	Passos	Expandidos	Tempo(ms)
BFS	True	12	12	36	~0.25
DFS	True	18	18	19	~0.08
UCS	True	12	12	46	~0.19
Gulosa	True	12	12	13	~0.09
A*	True	12	12	36	~0.16

Conclusoes: BFS, UCS, Gulosa e A encontram o caminho otimo (12 passos). DFS e mais rapido mas encontra caminho subotimo (18 passos). A e Gulosa expandem menos nos usando a heuristica Manhattan.

6.2 Busca Local - Lab2 (com coletas)

Os algoritmos de busca local otimizam a ordem de visitacao das coletas (problema combinatorial):

Algoritmo	Melhor Custo	Pior Custo	Media	Iteracoes
Hill-Climbing	~13	~18	~15	~30 exec.
Simulated Annealing	~13	~15	~14	~20 exec.
Algoritmo Genetico	~13	~14	~13	200 ger.

Conclusoes: Hill-Climbing e o mais rapido mas fica preso em otimos locais. SA escapa com probabilidade, obtendo resultados mais consistentes. GA converge ao otimo global com mais execucoes.

6.3 Busca Online - Lab1 (mapa desconhecido)

Algoritmo	Sucesso	Movimentos	Revisitas	Razao online/offline
-----------	---------	------------	-----------	----------------------

Replanning A*	True	~14-18	~2-6	~1.2-1.5
Online DFS	True	~20-40	~8-20	~1.7-3.0

Conclusões: Replanning A* se aproxima mais do ótimo offline ao replaneja com o mapa interno atualizado. Online DFS explora mais do ambiente mas faz mais backtracking. Razão = 1.0 indica desempenho idêntico ao offline; razão > 1.0 e o custo de não conhecer o mapa.

6.4 Heurística de Manhattan

A heurística usada em todos os algoritmos informados:

```
h(pos) = |pos.linha - objetivo.linha| + |pos.coluna - objetivo.coluna|
```

- * Admissível: nunca superestima o custo real (garante otimalidade do A*)
- * Consistente: satisfaz a desigualdade triangular ($h(n) \leq c(n, n') + h(n')$)
- * Eficiente: $O(1)$ por consulta

Para o problema com coletas, a heurística é estendida:

```
h((pos, C_r)) = max{ manhattan(pos, d) | d em C_r U {B} }
```

7. Visualização

O projeto inclui uma visualização web interativa gerada pelo script `gerar_html.py`. Para gerar:

```
python tp01/src/gerar_html.py
```

O arquivo `visualizacao.html` resultante pode ser aberto em qualquer navegador e contém três abas:

- * Busca Clássica: animação passo a passo dos 5 algoritmos em paralelo, mostrando fronteira (amarelo), explorados (azul) e caminho final (laranja). Controles de velocidade e reinício.
- * Busca Local: animação frame a frame mostrando o caminho no labirinto para cada execução (permutação) do HC, SA e GA, além de cards com métricas comparativas (melhor, pior, média, iterações).
- * Busca Online: animação revelando o mapa progressivamente - células desconhecidas (preto), reveladas livres (branco), paredes (cinza), agente (roxo), trilha (ciano). Exibe razão online/offline ao final.

Todos os 7 mapas (lab1-lab7) estão disponíveis via botões no topo da página.

8. Auditoria de Uso de IA

8.1 Ferramentas Utilizadas

Ferramenta	Finalidade
Claude (Anthropic)	Implementação, organização de resultados

8.2 Semana 1 - Busca Classica

Uso de IA: Nenhum para a implementacao dos algoritmos.

Os cinco algoritmos de busca classica foram adaptados diretamente do código fornecido pelo professor na ATV02. As adaptacoes (reorganizacao em arquivos independentes, extensao para coletas, adicao de metricas) foram feitas pela equipe sem auxilio de IA.

8.3 Semana 2 - Busca Local

Uso de IA: Assistencia na implementacao e geracao de ideias.

A IA auxiliou na estruturacao da representacao por permutacao de coletas, na definicao da vizinhanca por swap para HC, no esquema de resfriamento do SA e na estrutura de selecao/cruzamento/mutacao do GA. Em todos os casos, a equipe revisou, testou e ajustou o código gerado.

A IA tambem foi usada para gerar ideias sobre qual representacao de estado usar e como calcular a funcao de custo de forma eficiente com distancias pre-computadas.

8.4 Semana 3 - Busca Online

Uso de IA: Assistencia na implementacao.

A IA auxiliou na estruturacao do loop de replanejamento do Replanning A*, no backtracking sistematico do Online DFS e na classe LabirintoOnline que simula a percepcao parcial.

8.5 Visualizacao Web

Uso de IA: Geracao quase integral.

O código HTML/CSS/JavaScript da visualizacao foi gerado com forte assistencia de IA. A equipe descreveu o comportamento desejado e revisou/corrigiu o resultado no navegador.

8.6 Organizacao dos Resultados

A IA auxiliou na sugestao de quais metricas comparar, na estruturacao do texto do relatorio e na organizacao de tabelas comparativas.

8.7 O que NAO foi feito por IA

- * Definicao do problema e formulacao formal (PEAS, espaco de estados, heurísticas)
- * Escolha dos mapas de teste e criacao dos labirintos
- * Execucao dos experimentos e coleta dos dados
- * Analise critica dos resultados
- * Decisoes de projeto (representacao de estado com frozenset, raio de percepcao = 1)

9. Conclusao

O TP01 implementa com sucesso as tres abordagens de busca em labirinto, demonstrando as diferencas fundamentais entre elas:

Busca Classica resolve o problema de forma otima (A^* , UCS) ou completa (BFS) quando o mapa e conhecido, ao custo de manter uma fronteira na memoria.

Busca Local trata o problema como otimizacao combinatorial, sendo eficiente em memoria ($O(1)$) mas sujeita a otimos locais. O Algoritmo Genetico apresentou os melhores resultados medios para a ordem de coletas.

Busca Online permite que o agente aja em ambientes desconhecidos, pagando um custo extra (razao > 1.0) em relacao ao otimo offline. O Replanning A^* se mostrou mais eficiente que o Online DFS, aproximando-se do otimo offline com poucas revisitas.

A visualizacao web permite observar de forma intuitiva as diferencas de comportamento entre os algoritmos, sendo um recurso valioso para analise e apresentacao dos resultados.

Data de entrega: Junho de 2026

Disciplina: CSI457 - Agentes de Inteligencia Artificial

Instituicao: UFOP - Universidade Federal de Ouro Preto